Secure-by-Default Patterns for LLM, Agent, RAG, and MCP Systems

"Secure by default" means the *easiest* way to build the system is also a safe way: controls are on unless deliberately relaxed, and relaxing them is explicit and reviewable. Below are default-on patterns per surface.

LLM integration

- **Untrusted-input mindset.** Every prompt component (user, retrieved, tool, prior turn) is untrusted until proven otherwise. Normalize input (NFKC, strip zero-width) before any matching.
- **Secrets out of prompts.** Never put API keys or sensitive config in the system prompt (prevents LLM07). Pull secrets at execution from a vault.
- **Output is untrusted too.** Never `eval`/exec/render model output; encode it for its sink; validate against a schema; redact secrets/PII on the way out.
- **Bounded consumption.** Token, request-size, and per-principal rate/cost caps on by default.

Agentic workflows

- **Default deny on tools.** Allow-list `(tool, resource)`; scope to the end user.
- **Budgeted loops.** Step/tool-call/cost caps with hard termination.
- **HITL for irreversible actions** out of the box; opt-out requires sign-off.
- **Sandboxed, egress-denied execution** for any tool that runs code or fetches URLs.

RAG pipelines

- **Provenance allow-list.** Only ingest from approved sources; record source per chunk.
- **Spotlight retrieved content.** Fence chunks in unforgeable, content-hashed delimiters and tag them as data, never instructions (defeats indirect injection).
- **Quarantine scoring.** Flag chunks that look like instructions; surface, don't silently drop.
- **Per-user retrieval scoping.** Filter the vector store by the requester's entitlements so the model can't retrieve what the user can't see (defeats the RAG confused-deputy and cross-tenant leakage, LLM08).
- **DLP on ingest and egress.** Classify and redact sensitive data entering the index or leaving in an answer.

```
from airte.guardrails import RAGContext
from airte.data_pipeline import DLPEngine
ctx = RAGContext(allowed_sources=frozenset({"handbook"}), max_quarantine=0.6)
context_block = ctx.build([(chunk, "handbook")])
answer = DLPEngine().redact(model_answer)
```

MCP server deployments

- **Vet and pin servers** (supply chain). Review tool descriptions as code.
- **Authenticated sessions, authorized tool calls** (per-principal scopes).
- **Validate/confine inputs:** schema-check args, confine paths, allow-list URLs.
- **Sandbox + deny egress** for the server runtime.
- **Audit-log every tool call.**
- **HITL for high-risk tools.**

`airte.secure_mcp_server.SecureToolRegistry` wires authz → rate limit → input validation → execution → output redaction → audit into one enforced path, so the secure way is the default way.

The default-on checklist

A new integration should start with: input normalization **on**, output encoding/redaction **on**, tool allow-list **empty (deny-all)**, egress **denied**, secrets **in vault**, rate/cost caps **set**, audit logging **on**, and a red-team suite **wired into CI**. Every relaxation is a reviewed, logged exception.